Project Reference & Technical Notes

Architecture · Roadmap · Python Package Plan · Research · References

Prepared by: **Fareed** · 03 April 2026

Table of Contents

1.	Project Overview
2.	System Architecture
3.	End-to-End Data Flow
4.	Codebase Structure & Module Guide
5.	Research Contribution: AdaptiveGrid Strategy
6.	What Was Discussed
7.	Python Package Plan: faceattend
8.	Action Checklist & Roadmap
9.	Challenges & Solutions
10.	Full Tech Stack Reference
11.	References
12.	Quick Commands Cheatsheet

1. Project Overview

The **Face Detection Attendance System** is a production-ready, AI-powered full-stack application that automates student attendance tracking using computer vision. A teacher uploads one or more group photographs of a classroom; the system automatically identifies which students are present, marks attendance, and notifies parents of absent students via email — with zero manual roll-call required.

Beyond attendance tracking, the project contains a fully instrumented **research framework** for systematic ablation studies comparing face recognition models, grid strategies, and preprocessing pipelines. The core academic contribution is the **AdaptiveGrid tiling strategy** — a novel approach to improving face detection accuracy in unconstrained classroom group photographs.

Purpose	Automated classroom attendance via face recognition from group photographs
Core Innovation	AdaptiveGrid tiling — image split into 3×3 and 4×4 grids; each tile processed independently to detect small faces that full-image processing misses
Architecture	Two Docker containers: Python/Flask inference backend (port 8000) + Next.js teacher dashboard (port 3000)
Current Status	Fully working end-to-end system — REST API, frontend dashboard, email notifications, and research framework are all operational
License	MIT License (model/LICENSE) — open source, free to use and modify
Tech Stack	Python 3.9+ · DeepFace (Facenet/ArcFace) · TensorFlow 2.18 · Flask 3.1 · Next.js · Docker
Dataset	Student registry in Data/dataset.csv — columns: name, uin, parent_email, image_path(s)

2. System Architecture

The system follows a microservices architecture. Two independent Docker containers communicate over HTTP. The teacher interacts only with the Next.js frontend; all heavy computation happens in the Python backend.

Service	Technology	Port	Role
Service	Technology	Port	Role

Service	Technology	Port	Role
model_api	Python · Flask · DeepFace	8000	Image processing, face recognition, email dispatch. Built from model/Dockerfile.
client_app	Next.js · TypeScript	3000	Teacher dashboard UI: image upload, result display, authentication. Built from client/Dockerfile.

docker-compose.yml defines both services and wires them together. The only required environment variable is NEXT_PUBLIC_MODEL_API_SERVER which tells the frontend where the Python API lives.

```
# Start full stack with one command:
docker-compose up

# Individual service URLs:
#   Frontend:  http://localhost:3000
#   API:       http://localhost:8000

# Key env vars:
#   NEXT_PUBLIC_MODEL_API_SERVER=http://192.168.0.107:8000   (frontend)
#   ALLOWED_ORIGINS=http://localhost:3000,...                  (backend CORS)
#   Email=gmail@gmail.com   Password=app_password             (backend .env)
```

3. End-to-End Data Flow

Step	Component	Action
Step	Component	Action
①	Next.js frontend	Teacher selects 1-6 classroom photos + enters name and subject. Sends multipart POST to /upload.
②	api/app.py	Flask receives images, saves to uploads/, creates timestamped output directory (output/{name}_{timestamp}/).
③	metrics/division.py	Each uploaded image is split into 3×3 (9 tiles) and 4×4 (16 tiles) grids. All tiles saved to output directory.
④	deepface_model/main.py	recognize_faces_in_directory() processes every tile using DeepFace (Facenet model). Computes cosine similarity against face_encodings.pkl (threshold = 0.70).
⑤	Deduplication	Best-match dictionary ensures each student is counted at most once, regardless of how many tiles they appear in.

Step	Component	Action
⑥	Data/dataset_copy.csv	Recognised name looked up to retrieve UIN and parent_email fields.
⑦	api/email_sending.py	send_attendance_emails() sends individual notification email to each recognised student's parent via Gmail SMTP (port 587, STARTTLS).
⑧	JSON response	List of {name, uin, parent_email} objects returned to frontend for display on dashboard.

4. Codebase Structure & Module Guide

Repository layout:

```
Face-detection-attendance-system/
  client/                          Next.js teacher dashboard
    src/app/
      (routes)/Login,OTP,Signup    Authentication pages
      api/upload/route.js          Proxy to Python backend
      api/auth/[...nextauth]/      NextAuth.js OAuth
      page.jsx                     Home / main upload page
    Dockerfile
  model/                           Python inference backend
    api/
      app.py                       Flask REST API (main entry point)
      email_sending.py             SMTP email notifications
    deepface_model/
      main.py                      Core recognition engine
      testing.py                   Test utilities
    metrics/
      division.py                  Grid tile generation (3x3 and 4x4)
    recognition/                   Legacy dlib-based pipeline
      training.py                  face_recognition library training
      recognition.py               Old recognition pipeline
      face_cache.py                Pickle-based encoding cache
    research/                      Research & ablation framework
      config.yaml                  All experiment parameters
      experiment_runner.py         Runs full ablation sweeps
      grid/grid_strategy.py        Advanced GridStrategy class
      evaluation/metrics.py        Precision/Recall/F1/FAR/FRR
      evaluation/visualizer.py     Result image annotation
      models/model_factory.py      DeepFace model wrapper
      preprocessing/enhancement.py CLAHE/Sharpen/Denoise
    Data/
      dataset.csv                  Student registry (training)
      dataset_copy.csv             Student registry (inference)
    images/                        Per-student training images
    test_folder/                   Test classroom images
```

```
  face_encodings.pkl            Cached trained embeddings
  requirements.txt
  Dockerfile
docker-compose.yml
```

Key module descriptions:

api/app.py	Flask REST API. Handles file upload (max 6 images), calls grid splitting and recognition, sanitises NaN parent_email values, triggers email sending, returns JSON.
deepface_model/main.py	Primary recognition engine. train_faces() builds mean embeddings from CSV training images. recognize_faces_in_directory() processes all tiles and returns matched students with metadata.
metrics/division.py	Lightweight split_image() using OpenCV. Called at inference time to generate 3x3 and 4x4 tile sets from each uploaded image.
research/grid/grid_strategy.py	Advanced GridStrategy class. Supports configurable overlap (0.0-0.9), multiple grid sizes, min tile size filtering, and full process_image() pipeline. Used by ablation framework.
research/evaluation/metrics.py	MetricsCalculator. Computes Precision, Recall, F1, Accuracy, FAR, FRR, and per-class metrics. Requires ground-truth JSON file.
research/config.yaml	Single YAML file controlling all experiment parameters: recognition model, detector backend, grid sizes, preprocessing method, deduplication strategy, output format.
api/email_sending.py	Reads Email/Password from .env via python-dotenv. Sends per-student notification via Gmail SMTP (smtp.gmail.com:587 + STARTTLS).
recognition/training.py	Legacy pipeline using face_recognition library (dlib-based). Pre-dates DeepFace integration. Kept for benchmark comparison.

5. Research Contribution: AdaptiveGrid Strategy

The core academic contribution is the **AdaptiveGrid tiling strategy**. Standard single-shot face detection on group photographs performs poorly because individual faces are small relative to the full image. By splitting each image into multiple tiles, each face occupies a proportionally larger area within its tile, yielding significantly higher detection and recognition accuracy. This is the key novelty of the paper.

Grid sizes	Default [3, 4] — image split into 3×3 (9 tiles) and 4×4 (16 tiles) grids

Original included	Full image also processed alongside tiles (include_original: true in config)
Total tiles	1 (original) + 9 (3×3) + 16 (4×4) = 26 tiles processed per uploaded image
Overlap support	0.0 = non-overlapping (default); up to 0.9 = highly overlapping tiles for faces at grid boundaries
Min tile size	Tiles smaller than 64 px in either dimension are automatically skipped
Deduplication	Name-based: each student counted once regardless of how many tiles detected them

Research framework capabilities (research/ directory):

- **Ablation study:** Automatic sweep over 8 recognition models (VGG-Face, Facenet, Facenet512, ArcFace, SFace, OpenFace, DeepID, Dlib) × 2 detectors (RetinaFace, MTCNN) × 5 grid configurations = 80+ experiment combinations
- **Preprocessing variants:** None, CLAHE, Sharpen, Denoise, Combined — fully configurable without code changes
- **Evaluation metrics:** Precision, Recall, F1-Score, Accuracy, FAR (False Accept Rate), FRR (False Reject Rate), TAR@FAR at [0.001, 0.01, 0.1]
- **Timing statistics:** Per-image inference time (mean, std, min, max in ms)
- **Visualiser:** Annotated result images with bounding boxes and labels
- **Output formats:** CSV, JSON, or both; configurable in config.yaml

Model	Embed. Dim	Notes
Model	Embed. Dim	Notes
Facenet	128	Production default — good balance of speed and accuracy
Facenet512	512	Research default (in config.yaml) — higher accuracy
VGG-Face	4096	Oxford model, high accuracy, slower inference
ArcFace	512	State-of-the-art angular margin loss; excellent on LFW/IJB-C benchmarks
SFace	128	Lightweight; strong for mobile/edge deployment
OpenFace	128	Open-source; good for academic reproducibility
DeepID	160	Early deep learning face recognition approach
Dlib	128	HOG+CNN hybrid; used in legacy recognition pipeline

6. What Was Discussed

6.1 The project is more than an attendance system.

While the system is presented primarily as an attendance tool, the codebase is a complete **face recognition research platform**. The research framework, evaluation metrics module, ablation study infrastructure, and configurable preprocessing pipeline collectively point toward a publishable academic contribution — both as an open-source tool on PyPI and as a research paper at a computer vision venue (CVPR, ICCV, ECCV workshops, or IEEE Access).

6.2 Python package idea.

The idea of converting this project into an installable Python package (pip install faceattend) was discussed. This would allow any school, college, or developer to integrate face-recognition-based attendance into their own systems with minimal setup. The codebase is already sufficiently modular.

- The project already has an MIT License — open source from day one
- Each module (recognition engine, grid strategy, API server, email) is independently importable
- DeepFace, Flask, and OpenCV are all standard, well-maintained libraries on PyPI
- The research framework adds academic credibility that few similar packages can match
- Publishing on PyPI makes the package globally accessible and potentially citeable

6.3 Career and academic value.

- **Research paper:** AdaptiveGrid approach + systematic ablation study = publishable paper. Target venues: CVPR/ICCV workshops, BMVC, IEEE T-BIOM, Pattern Recognition Letters
- **Open-source portfolio:** A published PyPI package with GitHub stars is a strong differentiator on a CV/resume
- **Community contributions:** Once public, users can add new model backends, preprocessing methods, and language support
- **Academic citations:** If researchers use the package in their work, it generates formal citations in literature

7. Python Package Plan: faceattend

7.1 Package name options:

faceattend	■ Best choice — short, descriptive, memorable, PyPI-friendly, imports cleanly
face-attend	■ Hyphenated variant — valid on PyPI but import uses underscores: import face_attend

deepattend	Highlights DeepFace backend; less descriptive of the attendance use-case
classattend	Broader scope — could include non-face methods in future

7.2 Proposed package structure:

```
faceattend/                        Repository root
  faceattend/                      Python package
    __init__.py                    Public API (expose key functions)
    recognition/
      __init__.py
      engine.py                    recognize_faces_in_directory()
      training.py                  train_faces()
    grid/
      __init__.py
      strategy.py                  GridStrategy, split_image()
    api/
      __init__.py
      server.py                    create_app() Flask factory function
      email_sending.py
    research/                      Optional - pip install faceattend[research]
    cli.py                         NEW: command-line interface
  tests/
    test_recognition.py
    test_grid.py
    test_api.py
  pyproject.toml                   NEW: package metadata + dependencies
  README.md                        Update with install guide + quickstart
  CHANGELOG.md                     NEW: version history
  LICENSE                          MIT (already exists)
```

7.3 pyproject.toml skeleton:

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.backends.legacy:build"

[project]
name = "faceattend"
version = "0.1.0"
description = "AI-powered face recognition attendance system"
authors = [{name = "Fareed", email = "your@email.com"}]
license = {text = "MIT"}
requires-python = ">=3.9"
keywords = ["face-recognition", "attendance", "deepface", "computer-vision"]

[project.dependencies]
    "deepface>=0.0.93",
    "flask>=3.0",
    "flask-cors>=5.0",
    "opencv-python>=4.8",
    "scikit-learn>=1.5",
    "pandas>=2.0",
```

```
    "numpy>=2.0",
    "python-dotenv",

[project.optional-dependencies]
research = ["matplotlib", "PyYAML", "reportlab"]
gpu = ["tensorflow-gpu"]

[project.scripts]
faceattend = "faceattend.cli:main"

[project.urls]
Repository = "https://github.com/Fareed95/Face-detection-attendance-system"
```

7.4 CLI interface design:

```
# Train face embeddings from CSV
faceattend train --csv Data/dataset.csv --output face_encodings.pkl

# Recognise faces in a single image or directory
faceattend recognize --image classroom.jpg --threshold 0.70
faceattend recognize --dir ./images/ --threshold 0.70

# Start the Flask API server
faceattend serve --host 0.0.0.0 --port 8000

# Run research ablation experiments
faceattend research --config research/config.yaml
```

7.5 Public Python API (faceattend/__init__.py):

```
# After: pip install faceattend
from faceattend import train_faces, recognize_faces_in_directory, create_app

# 1. Train (run once)
train_faces("Data/dataset.csv", output="face_encodings.pkl")

# 2. Recognise faces in directory of tiles
results = recognize_faces_in_directory("output/", threshold=0.70)
# results -> [{'name': 'Fareed', 'uin': '...', 'parent_email': '...'}, ...]

# 3. Launch API server
app = create_app()
app.run(host="0.0.0.0", port=8000)
```

8. Action Checklist & Roadmap

Phase 1 — Packaging Setup (1–2 days)		
TODO	**Create pyproject.toml**	*Package metadata, dependency list, entry_points, optional extras [research] and [gpu]*

TODO	**Restructure directories**	*Reorganise model/ into faceattend/ package with __init__.py at each level*
TODO	**Define clean public API**	*faceattend/__init__.py exposes: train_faces, recognize_faces_in_directory, split_image, create_app*
TODO	**Build CLI (cli.py)**	*Implement train / recognize / serve / research commands using argparse or fire library*
TODO	**Remove hardcoded paths**	*Replace all hardcoded paths with os.getenv() or function arguments with sensible defaults*

Phase 2 — Tests & CI (2–3 days)

TODO	**Write unit tests**	*pytest tests for train_faces(), recognize_faces_in_directory(), split_image(), GridStrategy*
TODO	**Set up GitHub Actions CI**	*.github/workflows/ci.yml — run lint and tests on every pull request*
TODO	**Set up auto PyPI publish**	*.github/workflows/publish.yml — publish to PyPI automatically when a git tag is pushed*
TODO	**Add pre-commit hooks**	*Black (formatter) + Ruff (linter) via .pre-commit-config.yaml*
TODO	**Code cleanup**	*Remove commented-out code blocks, unused imports, and debug print() statements*

Phase 3 — Documentation (1–2 days)

TODO	**Update README.md**	*Installation guide, quickstart, API reference, Docker setup, environment variables*
TODO	**Add CHANGELOG.md**	*v0.1.0 release notes in Keep a Changelog format*
TODO	**Add Google-style docstrings**	*All public functions and classes need proper docstrings for Sphinx auto-docs*
TODO	**CONTRIBUTING.md**	*How to contribute: fork, branch, test, PR process, code style guide*
TODO	**GitHub issue templates**	*.github/ISSUE_TEMPLATE/ — bug report + feature request templates*
TODO	**README shields.io badges**	*PyPI version, downloads, license, CI status, Python version badges*

Phase 4 — Publish to PyPI (1 day)

TODO	**Create PyPI account**	*Register at pypi.org, verify email, generate API token for CI/CD*

TODO	**Test on TestPyPI first**	*pip install build twine → python -m build → twine upload --repository testpypi dist/**
TODO	**Verify clean install**	*Fresh virtual environment: pip install faceattend → run faceattend --help*
TODO	**Publish to real PyPI**	*twine upload dist/* (or let GitHub Actions do it on tag push)*
TODO	**Create GitHub Release**	*Tag v0.1.0, write release notes, attach .whl artifact*
TODO	**Announce**	*Share on Reddit r/MachineLearning, LinkedIn, Twitter/X, Dev.to*

9. Challenges & Solutions

Challenge	Solution
Challenge	Solution
TensorFlow is a very heavy install (~500 MB)	Use optional extras: pip install faceattend installs CPU TF by default; pip install faceattend[gpu] installs tensorflow-gpu for GPU-accelerated inference.
DeepFace downloads model weights on first run	This is expected behaviour — weights are cached in ~/.deepface/ and only downloaded once. Document clearly in README. For offline deployment, pre-bundle the weights.
Email credentials security	Already using python-dotenv correctly. Add .env to .gitignore (verify this). Document required env vars (Email, Password) in README. Gmail requires App-Specific Password when 2FA is enabled.
Hardcoded file paths in source code	Replace all hardcoded paths (Data/dataset_copy.csv, face_encodings.pkl, etc.) with configurable function arguments that have sensible defaults. Create a config.py with defaults.
face_encodings.pkl not in repository	This is correct — the .pkl is gitignored. Users must run faceattend train first. Prominently document this as Step 1 in README.
Python version compatibility	Target Python 3.9+ (TensorFlow 2.18 minimum). Test matrix in GitHub Actions: Python 3.9, 3.10, 3.11.
NaN values in parent_email column	Already handled in sanitize_face_recognition_result() in app.py. Document expected CSV schema so users know parent_email is optional.

Challenge	Solution
Large test images and .pkl file in repo	Move test images to a GitHub Release asset or Git LFS. Keep repository lightweight for fast cloning and CI.

10. Full Tech Stack Reference

10.1 Python backend (model/requirements.txt)

Package	Version	Role
Package	Version	Role
deepface	0.0.93	Face detection + embedding extraction via Facenet, ArcFace, VGG-Face, etc.
tensorflow	2.18.0	Deep learning backend powering all DeepFace models
opencv-python	4.11.0	Image I/O, grid tile generation, preprocessing operations
scikit-learn	1.6.1	Cosine similarity for comparing face embeddings
Flask	3.1.0	REST API server
flask-cors	5.0.1	Cross-Origin Resource Sharing for Next.js → Flask requests
pandas	2.2.3	CSV data loading and student registry management
numpy	2.0.2	Numerical operations, embedding averaging across training images
python-dotenv	latest	Load Email/Password from .env file at runtime (never hardcode!)
mtcnn	1.0.0	Multi-Task Cascaded CNN face detector (one of the detector options)
retina-face	0.0.17	RetinaFace detector — default in research config.yaml
gunicorn	23.0.0	Production WSGI server (used in model/Dockerfile instead of Flask dev server)
reportlab	4.4.0	PDF generation for research reports
matplotlib	3.10.1	Experiment result charts and comparison plots
PyYAML	6.0.2	Parse research/config.yaml experiment configuration
keras	3.9.0	High-level neural network API (TensorFlow backend)

Package	Version	Role
pillow	11.1.0	Image manipulation utilities

10.2 Frontend (client/package.json)

Next.js	React full-stack framework — pages, API routes, server-side rendering
TypeScript	Type-safe JavaScript throughout the frontend codebase
Tailwind CSS	Utility-first CSS framework for responsive, mobile-friendly styling
shadcn/ui	Component library built on Radix UI (components.json present in repo)
NextAuth.js	Authentication — Login / OTP / Signup routes exist in src/app/(routes)/

10.3 Infrastructure

Docker	Containerises both services for consistent, reproducible deployment across environments
docker-compose	Orchestrates multi-container setup: docker-compose up starts everything with one command
gunicorn	Production WSGI server replacing Flask dev server inside the Python Docker container
Git / GitHub	Version control, issue tracking, pull requests, GitHub Actions CI/CD
MIT License	Open-source license — allows free use, modification, and distribution with attribution

11. References

11.1 Core research papers

FaceNet (CVPR 2015)	Schroff F., Kalenichenko D., Philbin J. FaceNet: A Unified Embedding for Face Recognition and Clustering. https://arxiv.org/abs/1503.03832
ArcFace (CVPR 2019)	Deng J. et al. ArcFace: Additive Angular Margin Loss for Deep Face Recognition. https://arxiv.org/abs/1801.07698
RetinaFace (CVPR 2020)	Deng J. et al. RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild. https://arxiv.org/abs/1905.00641

MTCNN (2016)	Zhang K. et al. Joint Face Detection and Alignment using Multi-task Cascaded CNNs. IEEE Signal Processing Letters. https://arxiv.org/abs/1604.02878
VGGFace (BMVC 2015)	Parkhi O.M., Vedaldi A., Zisserman A. Deep Face Recognition. https://www.robots.ox.ac.uk/~vgg/publications/2015/Parkhi15/
SFace (ICIP 2021)	Zhong Y. et al. SFace: Privacy-Friendly and Accurate Face Recognition using Synthetic Data. https://arxiv.org/abs/2206.10494
DeepFace Library	Serengil S.I., Ozpinar A. LightFace: A Hybrid Deep Face Recognition Framework. ASYU 2020. https://github.com/serengil/deepface

11.2 Benchmarks and datasets

LFW	Labeled Faces in the Wild — standard face verification benchmark. http://vis-www.cs.umass.edu/lfw/
IJB-C	IARPA Janus Benchmark-C — face recognition at scale in unconstrained settings. https://www.nist.gov/programs-projects/face-challenges
VGGFace2	Large-scale face recognition dataset (3.3M images, 9K identities). https://github.com/ox-vgg/vgg_face2
MS-Celeb-1M	Microsoft Celebrity dataset used to pre-train Facenet models.

11.3 Python packaging

Python Packaging Guide	Official guide to packaging and distributing Python projects. https://packaging.python.org/en/latest/
PEP 621	Specification for pyproject.toml project metadata. https://peps.python.org/pep-0621/
Setuptools	Primary build backend documentation. https://setuptools.pypa.io/en/latest/
Twine	Secure upload to PyPI: pip install twine. https://twine.readthedocs.io
TestPyPI	Test your package before real PyPI publication. https://test.pypi.org
GitHub Actions + PyPI	Automated publishing guide. https://docs.github.com/en/actions/use-cases-and-examples/publishing-packages/publishing-python-packages
Semantic Versioning	Version numbering standard (MAJOR.MINOR.PATCH). https://semver.org

11.4 Open-source and documentation tools

Choose a License	Compare open-source licenses (MIT already chosen ■). https://choosealicense.com
Keep a Changelog	Standard format for CHANGELOG.md. https://keepachangelog.com
Contributor Covenant	Code of conduct template for open-source projects. https://www.contributor-covenant.org
Read the Docs	Free documentation hosting that auto-builds from Sphinx docstrings. https://readthedocs.org
Shields.io	README badges: PyPI version, downloads, CI status, license. https://shields.io
pre-commit	Git hook framework to enforce code quality before commits. https://pre-commit.com

11.5 Development tools

Tool	Install Command	Purpose
Tool	Install Command	Purpose
build	pip install build	Build wheel + sdist: python -m build
twine	pip install twine	Upload to PyPI: twine upload dist/*
bump2version	pip install bump2version	Auto-increment version across all files on git tag
Black	pip install black	Opinionated Python code formatter
Ruff	pip install ruff	Extremely fast Python linter (replaces flake8 + isort)
pytest	pip install pytest	Testing framework
pytest-cov	pip install pytest-cov	Coverage reports: pytest --cov=faceattend
Sphinx	pip install sphinx	Auto-generate documentation from docstrings
mypy	pip install mypy	Static type checking for Python

12. Quick Commands Cheatsheet

Run the full stack

```
docker-compose up                              # Recommended: starts both services

# Manual (without Docker):
cd model && python api/app.py                # Backend  http://localhost:8000
cd client && npm run dev                     # Frontend http://localhost:3000
```

Train face embeddings

```
cd model
python deepface_model/main.py
# Reads: Data/dataset_copy.csv
# Writes: face_encodings.pkl
# Must run this BEFORE using the recognition system
```

Run research experiments

```
cd model
# 1. Edit research/config.yaml to configure models, grids, preprocessing
# 2. Ensure Data/ground_truth.json exists for metric computation
python research/experiment_runner.py
# Results saved to research/results/
```

Build and publish the Python package

```
pip install build twine
python -m build                               # Creates dist/*.whl + dist/*.tar.gz

# Test on TestPyPI first:
twine upload --repository testpypi dist/*
pip install --index-url https://test.pypi.org/simple/ faceattend

# Publish to real PyPI:
twine upload dist/*
```

API reference

GET /	Health check — returns welcome string
POST /upload	Upload 1–6 classroom images (multipart/form-data) Form fields: images (files), name (str), subject_name (str) Returns: {recognized_faces: [{name, uin, parent_email}, ...]}

Required environment variables (.env file)

```
# model/.env  (NEVER commit this file to Git)
Email=your_gmail_address@gmail.com
Password=your_16_char_app_specific_password
ALLOWED_ORIGINS=http://localhost:3000,http://192.168.0.107:3000

# Note: Gmail requires an App-Specific Password (not your main password)
# when 2-Factor Authentication is enabled on your Google account.
```

Key Reminders: *(1) Always run python deepface_model/main.py (or faceattend train) before using the system — it will not work without face_encodings.pkl. (2) Gmail requires an App-Specific Password when*

2FA is enabled. (3) The research metrics framework needs Data/ground_truth.json to compute Precision/Recall/F1; without it only raw recognition results are returned.